

Project_2_Final_Working_Version_test

March 3, 2025

Utilizing Various Machine Learning Models to Classify Emotions Based Off Facial Expressions - Cassandra Morgan

For my project I decided to see if I could create a simple machine learning model to correctly classify the emotions displayed on human faces through analyzing and categorizing images of people and splitting them into 6 distinct categories: angry, fearful, happy, neutral, sad or surprised.

For this I used a dataset from Kaggle: <https://www.kaggle.com/datasets/ananthu017/emotion-detection-fer?resource=download>

I dropped the emotion category called “disgust” from the dataset and removed it from my list of classifiers in order to simplify the models. I also removed that category because it had only 1k observations, and the other datasets all had 3k+. I wanted to ensure that each category had a similar number of observations, but I did not want to limit myself to 1k observations per category, so I removed images classified as “disgust” from the dataset. For my models I used a train-test split for validation.

Analyzing the facial expressions of people in images to categorize their emotional state using machine learning has useful applications for analyzing images to make educational material for children or autistic people who struggle to identify emotions based off facial expressions. I used three separate models for this: a neural net, a random forest, and a kmeans clustering technique.

First I installed the necessary packages for creating my models and reading in the image data from the directories that the data were structured into.

For the first section, I chose to create a neural net.

Neural Net:

Next I imported the necessary modules.

```
[229]: import tensorflow as tf
from tensorflow.keras import layers
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import glob
from sklearn.preprocessing import LabelEncoder
import pandas as pd
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

Here, I created arrays to hold the names and associated index numbers.

```
[231]: class_names = ['Angry', 'Fearful', 'Happy', 'Neutral', 'Sad', 'Surprised']
       class_names_numbers = [int(0),int(1),int(2),int(3),int(4),int(5)]
```

Next I read in the image data from the directories. To do this, I used glob and cv2 to read in images file by file. The images were already randomly sorted within each directory, so I didn't have to sort them again. I truncated each array of images to be 3000 images long. I did this so that none of the categories would have more data than the rest. In the original data the happy category had 4k more images than the smallest category. It is known that neural nets do not do as well on unbalanced data, and I wanted to ensure it was not biased towards the happy category.

```
[233]: angry_image_list = []
       for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/train/angry/*.png'):
           angry_im = cv2.imread(filename)
           angry_image_list.append(angry_im)
       angry_image__array = np.array(angry_image_list)
       angry_image__array = angry_image__array[:3000]
       len(angry_image__array)
```

[233]: 3000

```
[234]: fearful_image_list = []
       for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/train/fearful/*.
       ↪png'):
           fearful_im= cv2.imread(filename)
           fearful_image_list.append(fearful_im)
       fearful_image__array = np.array(fearful_image_list)
       fearful_image__array = fearful_image__array[:3000]
       len(fearful_image__array)
```

[234]: 3000

```
[235]: happy_image_list = []
       for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/train/happy/*.png'):
           happy_im= cv2.imread(filename)
           happy_image_list.append(happy_im)
       happy_image__array = np.array(happy_image_list)
       happy_image__array = happy_image__array[:3000]
       len(happy_image__array)
```

[235]: 3000

```
[236]: neutral_image_list = []
       for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/train/neutral/*.
       ↪png'):
           neutral_im= cv2.imread(filename)
           neutral_image_list.append(neutral_im)
```

```
neutral_image__array = np.array(neutral_image_list)
neutral_image__array = neutral_image__array[:3000]
len(neutral_image__array)
```

[236]: 3000

```
[237]: sad_image_list = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/train/sad/*.png'):
    sad_im= cv2.imread(filename)
    sad_image_list.append(sad_im)
sad_image__array = np.array(sad_image_list)
sad_image__array = sad_image__array[:3000]
len(sad_image__array)
```

[237]: 3000

```
[238]: surprised_image_list = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/train/surprised/*.
    ↪png'):
    surprised_im= cv2.imread(filename)
    surprised_image_list.append(surprised_im)
surprised_image__array = np.array(surprised_image_list)
surprised_image__array = surprised_image__array[:3000]
len(surprised_image__array)
```

[238]: 3000

Combining all the training data into one training set for use with models.

```
[240]: train_set = np.concatenate((angry_image__array, fearful_image__array,
    ↪happy_image__array, neutral_image__array, sad_image__array,
    ↪surprised_image__array))
train_set_labels = np.empty(18000)
print(len(train_set))
print(len(train_set_labels))
```

18000

18000

Assigning the true labels of emotions for the training data.

```
[242]: index_break_0 = len(angry_image__array)
index_break_1 = index_break_0 + len(fearful_image__array)
index_break_2 = index_break_1 + len(happy_image__array)
index_break_3 = index_break_2 + len(neutral_image__array)
index_break_4 = index_break_3 + len(sad_image__array)
index_break_5 = index_break_4 + len(surprised_image__array)
```

```

train_set_labels[0:index_break_0] = 0
train_set_labels[index_break_0:index_break_1] = 1
train_set_labels[index_break_1:index_break_2] = 2
train_set_labels[index_break_2:index_break_3] = 3
train_set_labels[index_break_3:index_break_4] = 4
train_set_labels[index_break_4:index_break_5] = 5

```

I looked at the shape of the training set.

```

[244]: print('number of images:', len(train_set))

#checking type of image
print('type:', type(train_set[0]))

#shape of the first image
print('shape:', train_set[0].shape)

#what are minimum and maximum values in the entire train_images
print('image value range:', (np.min(train_set), np.max(train_set)))

```

```

number of images: 18000
type: <class 'numpy.ndarray'>
shape: (48, 48, 3)
image value range: (0, 255)

```

```

[245]: print('number of labels:', len(train_set_labels))
print('type:', type(train_set_labels))
print('shape:', train_set_labels.shape)
print('set of label values:', set(train_set_labels))

```

```

number of labels: 18000
type: <class 'numpy.ndarray'>
shape: (18000,)
set of label values: {0.0, 1.0, 2.0, 3.0, 4.0, 5.0}

```

Next, I read in the test data and created arrays from the images.

```

[247]: angry_image_list_test = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/test/angry/*.png'):
    angry_im_test = cv2.imread(filename)
    angry_image_list_test.append(angry_im_test)
angry_image__array_test = np.array(angry_image_list_test)
len(angry_image__array_test)

```

```

[247]: 958

```

```

[248]: fearful_image_list_test = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/test/fearful/*.
↳png'):

```

```
fearful_im_test = cv2.imread(filename)
fearful_image_list_test.append(fearful_im_test)
fearful_image__array_test = np.array(fearful_image_list_test)
len(fearful_image__array_test)
```

[248]: 1024

```
[249]: happy_image_list_test = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/test/happy/*.png'):
    happy_im_test = cv2.imread(filename)
    happy_image_list_test.append(happy_im_test)
happy_image__array_test = np.array(happy_image_list_test)
len(happy_image__array_test)
```

[249]: 1774

```
[250]: neutral_image_list_test = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/test/neutral/*.
↳png'):
    neutral_im_test = cv2.imread(filename)
    neutral_image_list_test.append(neutral_im_test)
neutral_image__array_test = np.array(neutral_image_list_test)
len(neutral_image__array_test)
```

[250]: 1233

```
[251]: sad_image_list_test = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/test/sad/*.png'):
    sad_im_test = cv2.imread(filename)
    sad_image_list_test.append(sad_im_test)
sad_image__array_test = np.array(sad_image_list_test)
len(sad_image__array_test)
```

[251]: 1247

```
[252]: surprised_image_list_test = []
for filename in glob.glob('D:/Adult Stuff/DAT_402/Project_2/test/surprised/*.
↳png'):
    surprised_im_test = cv2.imread(filename)
    surprised_image_list_test.append(surprised_im_test)
surprised_image__array_test = np.array(surprised_image_list_test)
len(surprised_image__array_test)
```

[252]: 831

Combining the test data.

```
[254]: test_set = np.concatenate((angry_image__array_test, fearful_image__array_test,
    ↪happy_image__array_test, neutral_image__array_test, sad_image__array_test,
    ↪surprised_image__array_test))
test_set_labels = np.zeros(7067, dtype=int)
print(len(test_set))
print(len(test_set_labels))
```

7067

7067

Assigning the true labels of emotions for the training data.

```
[256]: index_break_0_test = len(angry_image__array_test)
index_break_1_test = index_break_0_test + len(fearful_image__array_test)
index_break_2_test = index_break_1_test + len(happy_image__array_test)
index_break_3_test = index_break_2_test + len(neutral_image__array_test)
index_break_4_test = index_break_3_test + len(sad_image__array_test)
index_break_5_test = index_break_4_test + len(surprised_image__array_test)

test_set_labels[0:index_break_0_test] = 0
test_set_labels[index_break_0_test:index_break_1_test] = 1
test_set_labels[index_break_1_test:index_break_2_test] = 2
test_set_labels[index_break_2_test:index_break_3_test] = 3
test_set_labels[index_break_3_test:index_break_4_test] = 4
test_set_labels[index_break_4_test:index_break_5_test] = 5
```

```
[257]: print('test_images shape:', test_set.shape)
print('image value range:', (np.min(test_set), np.max(test_set)))

print('test_labels shape:', test_set_labels.shape)
print('set of label values', set(test_set_labels))
```

test_images shape: (7067, 48, 48, 3)

image value range: (0, 255)

test_labels shape: (7067,)

set of label values {0, 1, 2, 3, 4, 5}

```
[258]: print('test_images shape:', train_set.shape)
print('image value range:', (np.min(train_set), np.max(train_set)))

print('test_labels shape:', train_set_labels.shape)
print('set of label values', set(train_set_labels))
```

test_images shape: (18000, 48, 48, 3)

image value range: (0, 255)

test_labels shape: (18000,)

set of label values {0.0, 1.0, 2.0, 3.0, 4.0, 5.0}

Concepts from: HW4

This next code chunk is for plotting an image and its estimate for the label along with the true label. This code works for neural nets.

```
[260]: def plot_image(i, predictions_array, true_label, img):
    true_label, img = true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

    plt.imshow(img, cmap=plt.cm.binary)

    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = (0,0.6,0)
    else:
        color = 'red'

    plt.xlabel("{} {:2.0f}% ({})".format(class_names[predicted_label],
                                        100*np.max(predictions_array),
                                        class_names[true_label]),
              color=color)

def plot_value_array(i, predictions_array, true_label):
    true_label = true_label[i]
    plt.grid(False)
    plt.xticks(range(6))
    plt.yticks([])
    thisplot = plt.bar(range(6), predictions_array, color=(0.7,0.7,0.7))
    ↪#paint all bars in gray
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

    thisplot[predicted_label].set_color('red') #over-paint the predicted (i.e. ↪
    ↪tallest) bar in red
    thisplot[true_label].set_color((0,0.8,0)) #over-paint the bar for the true ↪
    ↪label in green
```

This code does the same plotting procedure but for non-neural nets.

```
[262]: def plot_image_2 (i, predictions_array, true_label_array, img):
    true_label, img = true_label_array[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

    plt.imshow(img, cmap=plt.cm.binary)

    if predictions_array[i] == true_label_array[i]:
```

```

        color = (0,0.6,0)
    else:
        color = 'red'

    plt.xlabel("{} ({}).format(class_names[int(predictions_array[i])],
                                class_names[true_label_array[i]]),
                                color=color)

def plot_value_array_2(i, predictions_array, true_label):
    true_label = true_label[i]
    plt.grid(False)
    plt.xticks(range(6))
    plt.yticks([])
    thisplot = plt.bar(range(6), predictions_array, color=(0.7,0.7,0.7))
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

    thisplot[predicted_label].set_color('red') #over-paint the predicted (i.e. ↵
↵tallest) bar in red
    thisplot[true_label].set_color((0,0.8,0)) #over-paint the bar for the true ↵
↵label in green

```

I scaled the training and test data because it has to be between 0 and 1 for the neural net. The values ranged from 0 to 255 before because they were describing pixels.

```

[264]: train_set_nn = train_set / 255
       test_set_nn = test_set / 255

```

Next, I built the model for the neural net. I used a convolution neural net with relu for the first two activation functions, and my output layer's activation function was softmax.

```

[266]: np.random.seed(234) #set the seed in numpy
       tf.random.set_seed(2345) #set the seed in tensorflow

mynet = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(48, 48, ↵
↵3)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(6, activation='softmax')
], name = 'mynet')

```

C:\Users\Damian\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential

models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

For my optimizer during the compiling step, I used 'adam' because it is good for image classification.

```
[268]: mynet.compile(optimizer='adam',  
                  loss='sparse_categorical_crossentropy',  
                  metrics=['accuracy'])
```

Next I fit the model for 1 through 50 epochs and recorded the accuracy, loss, validation accuracy, and validation loss.

```
[270]: history = mynet.fit(x=train_set_nn, y=train_set_labels,   
    ↪ validation_data=(test_set_nn, test_set_labels), epochs=50, batch_size=32)
```

Epoch 1/50

```
563/563          6s 9ms/step -  
accuracy: 0.2831 - loss: 1.6869 - val_accuracy: 0.4172 - val_loss: 1.4866
```

Epoch 2/50

```
563/563          4s 8ms/step -  
accuracy: 0.4433 - loss: 1.4145 - val_accuracy: 0.4794 - val_loss: 1.3472
```

Epoch 3/50

```
563/563          4s 7ms/step -  
accuracy: 0.4931 - loss: 1.3086 - val_accuracy: 0.4889 - val_loss: 1.3151
```

Epoch 4/50

```
563/563          4s 8ms/step -  
accuracy: 0.5353 - loss: 1.2223 - val_accuracy: 0.4920 - val_loss: 1.3106
```

Epoch 5/50

```
563/563          4s 7ms/step -  
accuracy: 0.5633 - loss: 1.1479 - val_accuracy: 0.4941 - val_loss: 1.3385
```

Epoch 6/50

```
563/563          4s 7ms/step -  
accuracy: 0.5954 - loss: 1.0732 - val_accuracy: 0.4961 - val_loss: 1.3530
```

Epoch 7/50

```
563/563          4s 7ms/step -  
accuracy: 0.6291 - loss: 1.0027 - val_accuracy: 0.4921 - val_loss: 1.3826
```

Epoch 8/50

```
563/563          5s 9ms/step -  
accuracy: 0.6580 - loss: 0.9369 - val_accuracy: 0.4926 - val_loss: 1.4219
```

Epoch 9/50

```
563/563          4s 7ms/step -  
accuracy: 0.6832 - loss: 0.8746 - val_accuracy: 0.4869 - val_loss: 1.4715
```

Epoch 10/50

```
563/563          4s 7ms/step -  
accuracy: 0.7033 - loss: 0.8225 - val_accuracy: 0.4909 - val_loss: 1.5395
```

Epoch 11/50

```
563/563          4s 7ms/step -  
accuracy: 0.7209 - loss: 0.7744 - val_accuracy: 0.4900 - val_loss: 1.6198
```

Epoch 12/50
563/563 4s 8ms/step -
accuracy: 0.7403 - loss: 0.7275 - val_accuracy: 0.4855 - val_loss: 1.7309

Epoch 13/50
563/563 4s 7ms/step -
accuracy: 0.7531 - loss: 0.6885 - val_accuracy: 0.4796 - val_loss: 1.8449

Epoch 14/50
563/563 4s 8ms/step -
accuracy: 0.7671 - loss: 0.6532 - val_accuracy: 0.4745 - val_loss: 1.9312

Epoch 15/50
563/563 4s 8ms/step -
accuracy: 0.7833 - loss: 0.6132 - val_accuracy: 0.4701 - val_loss: 2.0210

Epoch 16/50
563/563 4s 8ms/step -
accuracy: 0.7934 - loss: 0.5839 - val_accuracy: 0.4648 - val_loss: 2.1189

Epoch 17/50
563/563 5s 8ms/step -
accuracy: 0.8060 - loss: 0.5543 - val_accuracy: 0.4619 - val_loss: 2.2342

Epoch 18/50
563/563 5s 8ms/step -
accuracy: 0.8149 - loss: 0.5304 - val_accuracy: 0.4626 - val_loss: 2.3572

Epoch 19/50
563/563 5s 8ms/step -
accuracy: 0.8246 - loss: 0.5065 - val_accuracy: 0.4612 - val_loss: 2.5177

Epoch 20/50
563/563 5s 8ms/step -
accuracy: 0.8351 - loss: 0.4853 - val_accuracy: 0.4500 - val_loss: 2.6644

Epoch 21/50
563/563 4s 8ms/step -
accuracy: 0.8370 - loss: 0.4677 - val_accuracy: 0.4497 - val_loss: 2.7765

Epoch 22/50
563/563 5s 8ms/step -
accuracy: 0.8402 - loss: 0.4566 - val_accuracy: 0.4527 - val_loss: 2.9204

Epoch 23/50
563/563 5s 9ms/step -
accuracy: 0.8401 - loss: 0.4454 - val_accuracy: 0.4521 - val_loss: 3.0072

Epoch 24/50
563/563 5s 8ms/step -
accuracy: 0.8450 - loss: 0.4284 - val_accuracy: 0.4563 - val_loss: 3.0568

Epoch 25/50
563/563 4s 8ms/step -
accuracy: 0.8562 - loss: 0.4055 - val_accuracy: 0.4596 - val_loss: 3.1014

Epoch 26/50
563/563 4s 8ms/step -
accuracy: 0.8622 - loss: 0.3915 - val_accuracy: 0.4585 - val_loss: 3.2696

Epoch 27/50
563/563 5s 8ms/step -
accuracy: 0.8680 - loss: 0.3733 - val_accuracy: 0.4614 - val_loss: 3.4071

Epoch 28/50
563/563 5s 8ms/step -
accuracy: 0.8790 - loss: 0.3451 - val_accuracy: 0.4525 - val_loss: 3.5848
Epoch 29/50
563/563 5s 9ms/step -
accuracy: 0.8793 - loss: 0.3366 - val_accuracy: 0.4503 - val_loss: 3.7583
Epoch 30/50
563/563 4s 8ms/step -
accuracy: 0.8848 - loss: 0.3198 - val_accuracy: 0.4520 - val_loss: 3.8161
Epoch 31/50
563/563 4s 8ms/step -
accuracy: 0.8834 - loss: 0.3162 - val_accuracy: 0.4405 - val_loss: 3.9740
Epoch 32/50
563/563 4s 8ms/step -
accuracy: 0.8831 - loss: 0.3152 - val_accuracy: 0.4259 - val_loss: 4.2248
Epoch 33/50
563/563 4s 8ms/step -
accuracy: 0.8932 - loss: 0.2920 - val_accuracy: 0.4358 - val_loss: 4.2285
Epoch 34/50
563/563 4s 8ms/step -
accuracy: 0.9046 - loss: 0.2697 - val_accuracy: 0.4351 - val_loss: 4.4619
Epoch 35/50
563/563 5s 8ms/step -
accuracy: 0.9046 - loss: 0.2600 - val_accuracy: 0.4456 - val_loss: 4.4597
Epoch 36/50
563/563 5s 8ms/step -
accuracy: 0.9112 - loss: 0.2495 - val_accuracy: 0.4517 - val_loss: 4.5167
Epoch 37/50
563/563 4s 8ms/step -
accuracy: 0.9090 - loss: 0.2503 - val_accuracy: 0.4442 - val_loss: 4.6264
Epoch 38/50
563/563 4s 8ms/step -
accuracy: 0.9109 - loss: 0.2455 - val_accuracy: 0.4494 - val_loss: 4.6317
Epoch 39/50
563/563 4s 8ms/step -
accuracy: 0.9091 - loss: 0.2426 - val_accuracy: 0.4446 - val_loss: 4.7567
Epoch 40/50
563/563 4s 8ms/step -
accuracy: 0.9175 - loss: 0.2271 - val_accuracy: 0.4432 - val_loss: 4.8936
Epoch 41/50
563/563 5s 9ms/step -
accuracy: 0.9225 - loss: 0.2086 - val_accuracy: 0.4446 - val_loss: 5.0953
Epoch 42/50
563/563 5s 9ms/step -
accuracy: 0.9217 - loss: 0.2049 - val_accuracy: 0.4395 - val_loss: 5.1799
Epoch 43/50
563/563 4s 8ms/step -
accuracy: 0.9327 - loss: 0.1885 - val_accuracy: 0.4348 - val_loss: 5.3831

```

Epoch 44/50
563/563          5s 8ms/step -
accuracy: 0.9322 - loss: 0.1805 - val_accuracy: 0.4382 - val_loss: 5.3172
Epoch 45/50
563/563          5s 8ms/step -
accuracy: 0.9318 - loss: 0.1790 - val_accuracy: 0.4377 - val_loss: 5.4972
Epoch 46/50
563/563          4s 8ms/step -
accuracy: 0.9349 - loss: 0.1792 - val_accuracy: 0.4392 - val_loss: 5.5724
Epoch 47/50
563/563          5s 9ms/step -
accuracy: 0.9349 - loss: 0.1762 - val_accuracy: 0.4279 - val_loss: 5.7103
Epoch 48/50
563/563          5s 8ms/step -
accuracy: 0.9302 - loss: 0.1870 - val_accuracy: 0.4442 - val_loss: 5.5829
Epoch 49/50
563/563          5s 8ms/step -
accuracy: 0.9322 - loss: 0.1819 - val_accuracy: 0.4469 - val_loss: 5.6933
Epoch 50/50
563/563          5s 9ms/step -
accuracy: 0.9353 - loss: 0.1722 - val_accuracy: 0.4464 - val_loss: 5.8734

```

```

[271]: training_accuracy = history.history['accuracy']
       validation_accuracy = history.history['val_accuracy']
       epoch_number = np.arange(1,51)

```

Here I plotted the training accuracy and validation accuracy against the number of epochs. The training accuracy continues to as the epochs rise, but the validation accuracy levels off after about 6 epochs.

```

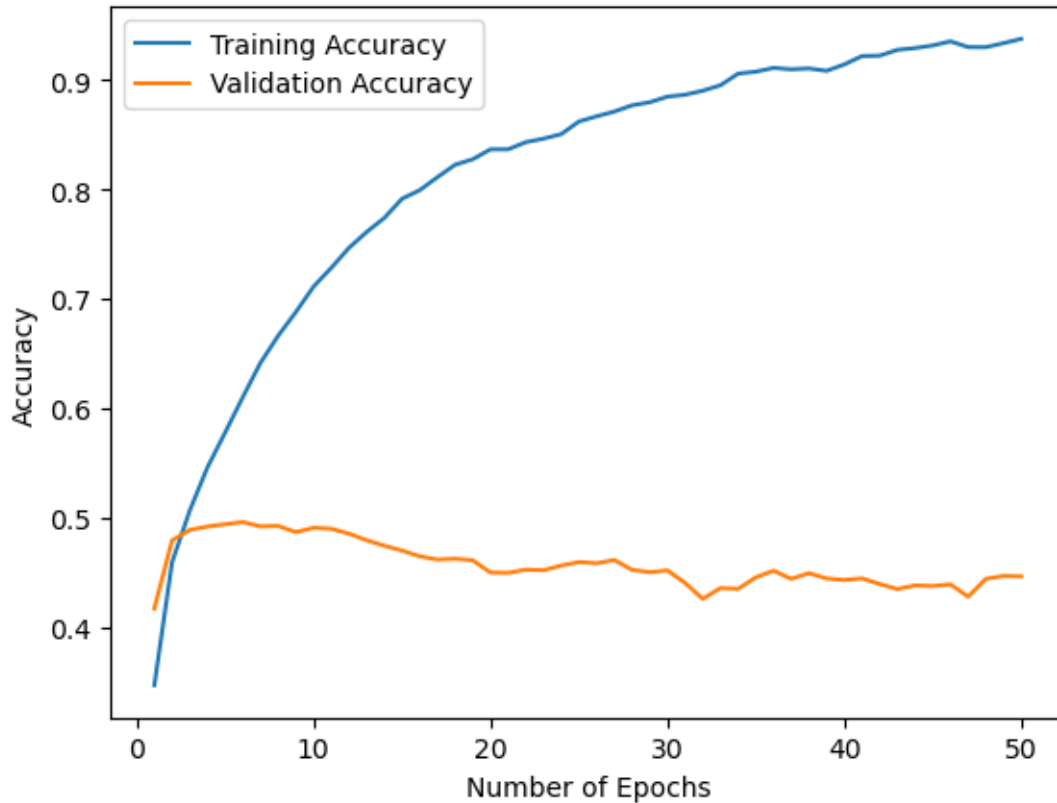
[273]: plt.plot(epoch_number, training_accuracy, label = 'Training Accuracy')
       plt.xlabel("Number of Epochs")
       plt.ylabel("Accuracy")
       plt.plot(epoch_number, validation_accuracy, label = 'Validation Accuracy')
       plt.legend()

```

```

[273]: <matplotlib.legend.Legend at 0x1a7dc8fed50>

```

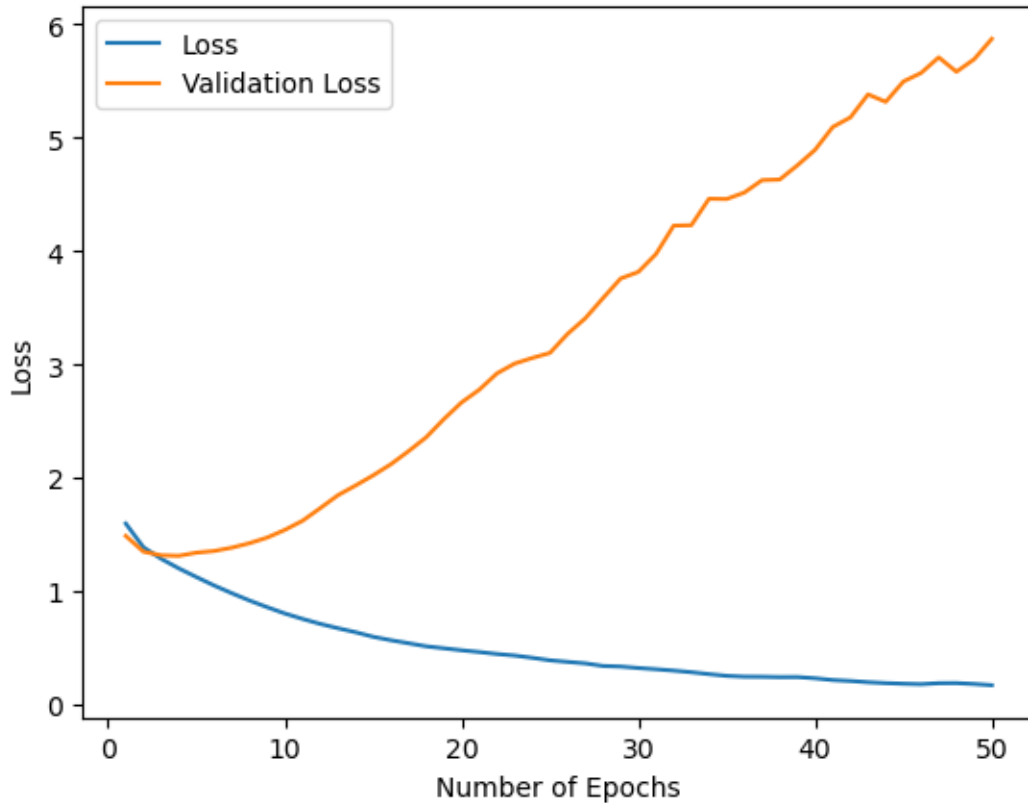


```
[274]: loss = history.history['loss']  
validation_loss = history.history['val_loss']
```

I plotted loss vs validation loss and found that to minimize the validation loss I want around 4 epochs. After 4 epochs the validation loss begins to increase. Due to this and the information about the accuracy, I chose 4 epochs for the fitting for my model.

```
[276]: plt.plot(epoch_number, loss, label = 'Loss')  
plt.xlabel("Number of Epochs")  
plt.ylabel("Loss")  
plt.plot(epoch_number, validation_loss, label = 'Validation Loss')  
plt.legend()
```

```
[276]: <matplotlib.legend.Legend at 0x1a7c5ac5010>
```



Here I fit the model again with 9 epochs.

```
[278]: history_2 = mynet.fit(x=train_set_nn, y=train_set_labels,
    ↪ validation_data=(test_set_nn, test_set_labels), epochs=4, batch_size=32)
```

Epoch 1/4

563/563 5s 8ms/step -

accuracy: 0.9378 - loss: 0.1682 - val_accuracy: 0.4384 - val_loss: 6.0342

Epoch 2/4

563/563 5s 8ms/step -

accuracy: 0.9402 - loss: 0.1644 - val_accuracy: 0.4371 - val_loss: 6.1767

Epoch 3/4

563/563 4s 8ms/step -

accuracy: 0.9395 - loss: 0.1592 - val_accuracy: 0.4365 - val_loss: 6.3397

Epoch 4/4

563/563 5s 9ms/step -

accuracy: 0.9339 - loss: 0.1706 - val_accuracy: 0.4455 - val_loss: 6.1566

Next, I calculated used my model to make predictions with the test data.

```
[280]: predictions_nn = mynet.predict(test_set_nn)
```

221/221

1s 2ms/step

```
[281]: predicted_labels_nn = np.zeros(len(predictions_nn))

for i in range(len(predictions_nn)):
    predicted_labels_nn[i] = np.argmax(predictions_nn[i])
```

Here I found that the accuracy for the test data predictions was about 45%, which is decent, but not ideal.

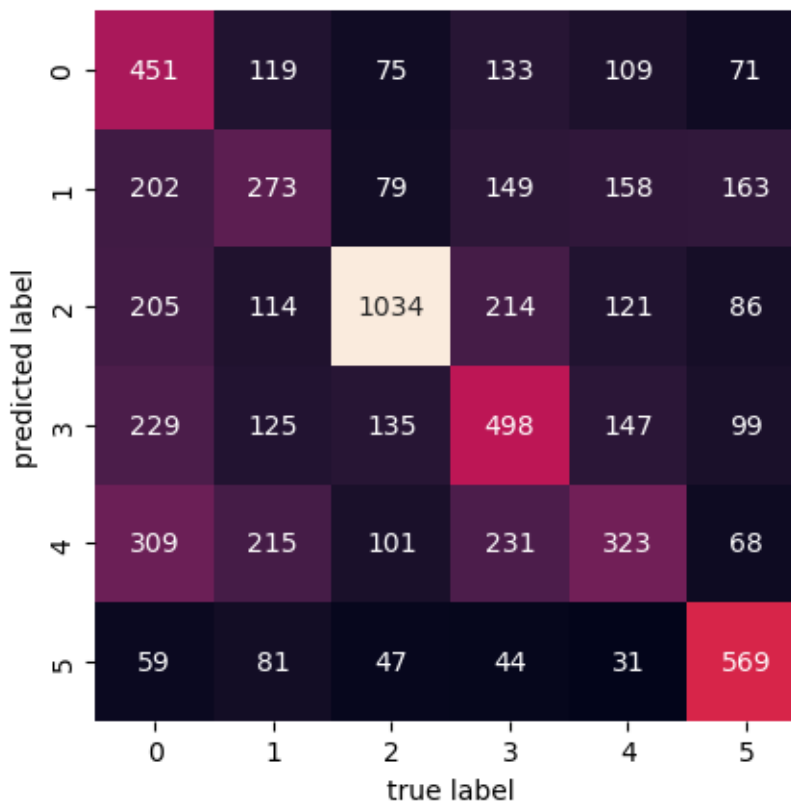
```
[283]: test_loss, test_acc = mynet.evaluate(test_set_nn, test_set_labels, verbose=2)
print("\nEmotional Classification Validation Accuracy for Neural Net Model:",
      test_acc)
```

221/221 - 1s - 2ms/step - accuracy: 0.4455 - loss: 6.1566

Emotional Classification Validation Accuracy for Neural Net Model:
0.44545069336891174

I created a confusion matrix using the predicted labels and the true labels for the test data.

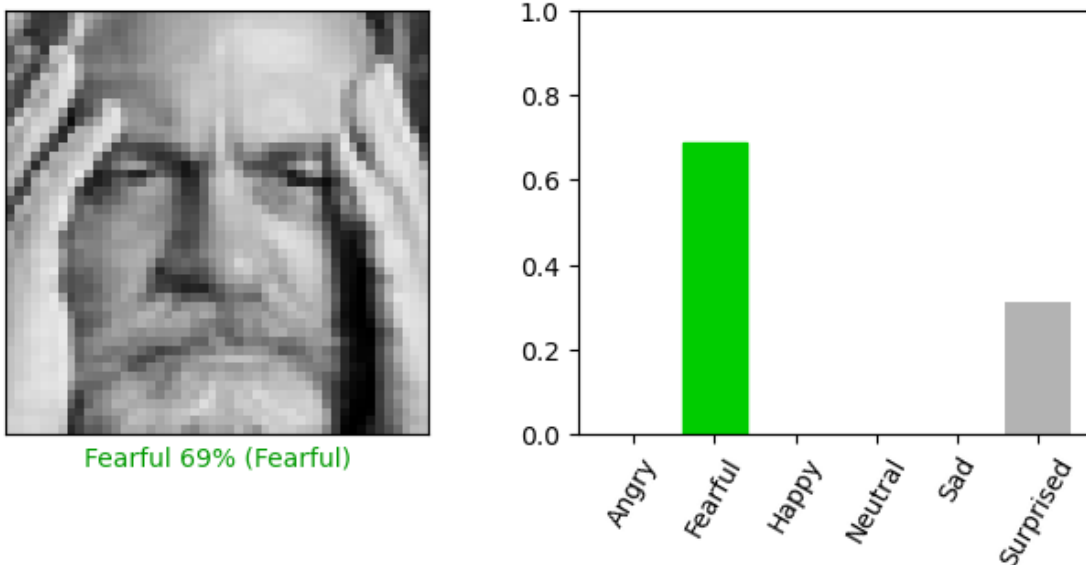
```
[285]: mat_nn = confusion_matrix(predicted_labels_nn, test_set_labels)
sns.heatmap(mat_nn.T, square=True, annot=True, fmt='d', cbar=False,
            xticklabels= class_names_numbers,
            yticklabels= class_names_numbers)
plt.xlabel('true label')
plt.ylabel('predicted label');
```



Testing the model on one of the images from the test set. It correctly guessed the expression of the image.

```
[287]: i = 1330

plt.figure(figsize=(8,3))
plt.subplot(1,2,1)
plot_image(i, predictions_nn[i], test_set_labels, test_set)
plt.subplot(1,2,2)
plot_value_array(i, predictions_nn[i], test_set_labels)
plt.xticks(range(6), class_names, rotation=60)
plt.yticks(0.2*np.arange(6))
plt.show()
```



Next up I used a Random Forest classification technique.

Random Forest:

Concepts From: <https://www.geeksforgeeks.org/random-forest-for-image-classification-using-opencv/>

I imported more important modules.

```
[292]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from skimage.feature import hog
import matplotlib.pyplot as plt
```



```
import os
import random
```

Extracting an image's hog features is a convenient way to simplify image data into easily usable chunks of data in order to simplify the creation of things like random forests. Here is a function that extracts the hog features.

```
[294]: def extract_hog_features(image):
        # Calculate HOG features
        hog_features = hog(image, orientations=9, pixels_per_cell=(8, 8),
                           cells_per_block=(2, 2), visualize=False, channel_axis = -1)
        return hog_features
```

This extracts hog features for input arrays.

```
[296]: def load_and_extract_features(array_rf):
        X_rf = []
        for i in range(len(array_rf)):
            hog_features = extract_hog_features(array_rf[i])
            X_rf.append(hog_features)
        return X_rf
```

This function trains random forests.

```
[298]: def train_random_forest(X_train_rf, y_train_rf):
        rf_classifier = RandomForestClassifier(n_estimators=1000, criterion='gini',
        ↪max_depth=5)
        rf_classifier.fit(X_train_rf, y_train_rf)
        return rf_classifier
```

Using the created function to extract the hog features of the training data.

```
[317]: emotion_train_X_rf = load_and_extract_features(train_set)
        emotion_train_y_rf = train_set_labels
```

Next, I used the above random forest function to train a random forest using the training data.

```
[334]: emotion_rf_classifier = train_random_forest(emotion_train_X_rf,
        ↪emotion_train_y_rf)
```

I performed the same hog extraction technique on the test set.

```
[336]: emotion_test_X_rf = load_and_extract_features(test_set)
        emotion_test_y_rf = test_set_labels
```

I then used my random forest to make predictions based off the test data. The accuracy of the random forest predictions based off the test set was nearly 40%. This means it is likely more accurate for out of sample data than the neural net.

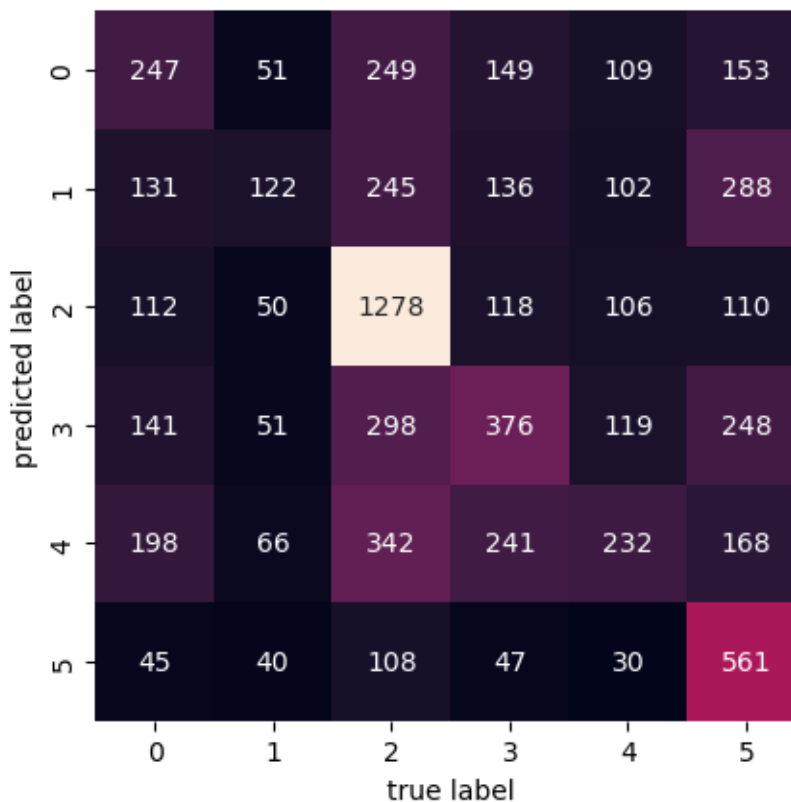
```
[338]: emotion_predictions_rf = emotion_rf_classifier.predict(emotion_test_X_rf)
emotion_accuracy_rf = accuracy_score(emotion_test_y_rf, emotion_predictions_rf)

print(f"Emotion Classification Validation Accuracy for Random Forest Model:␣
↪{emotion_accuracy_rf}")
```

Emotion Classification Validation Accuracy for Random Forest Model:
0.3984717701995189

Another confusion matrix but for the random forest predictions this time.

```
[340]: mat_rf = confusion_matrix(emotion_predictions_rf, test_set_labels)
sns.heatmap(mat_rf.T, square=True, annot=True, fmt='d', cbar=False,
            xticklabels= class_names_numbers,
            yticklabels= class_names_numbers)
plt.xlabel('true label')
plt.ylabel('predicted label');
```



I chose another random image to test, and this model correctly predicted the emotion label.

```
[342]: i = 3678
plt.figure(figsize=(8,3))
```

```
plt.subplot(1,2,1)
plot_image_2(i, emotion_predictions_rf, test_set_labels, test_set)
```



Happy (Happy)

K Clustering:

Concepts From: https://medium.com/@joel_34096/k-means-clustering-for-image-classification-a648f28bdc47

K-means clustering can only be used on arrays of particular dimensions. I needed to make my arrays dim 2 or less. So, I reshaped them.

```
[346]: train_set_kcl = train_set.reshape(len(train_set),-1)
test_set_kcl = test_set.reshape(len(test_set),-1)
```

```
[347]: print(train_set_kcl.shape)
print(test_set_kcl.shape)
```

```
(18000, 6912)
```

```
(7067, 6912)
```

```
[348]: from sklearn.cluster import MiniBatchKMeans
```

I created a minibatchkmeans model and fit it using the k clustering training set.

```
[350]: total_clusters = len(np.unique(test_set_labels))

kmeans = MiniBatchKMeans(n_clusters = total_clusters, batch_size = 2048)
# Fitting the model to training set
kmeans.fit(train_set_kcl)
```

```
[350]: MiniBatchKMeans(batch_size=2048, n_clusters=6)
```

```
[351]: kmeans.labels_
```

```
[351]: array([2, 1, 5, ..., 3, 0, 0])
```

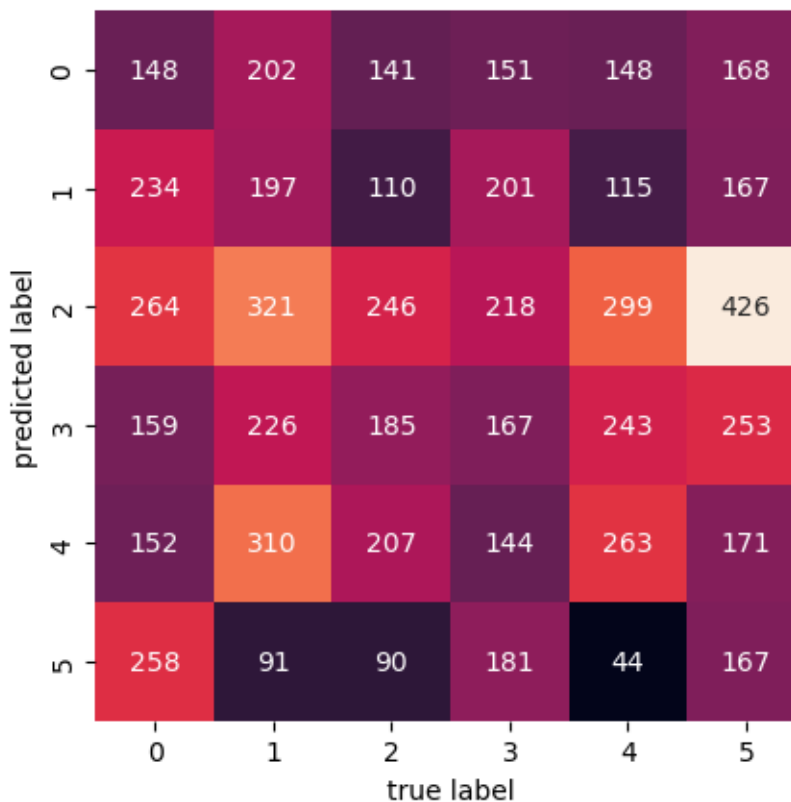
I calculated the accuracy for the predictions of the clustering model based off the testing data. It had an test set prediction accuracy of 17% which is particularly bad compared to the first two models.

```
[353]: emotion_predictions_kcl = kmeans.predict(test_set_kcl)
emotion_accuracy_kcl = accuracy_score(test_set_labels, emotion_predictions_kcl)
print(f"Emotion Classification Validation Accuracy For MiniBatchKmeans_
↳Clustering Model: {emotion_accuracy_kcl}")
```

Emotion Classification Validation Accuracy For MiniBatchKmeans Clustering Model:
0.16810527805292202

A confusion matrix for the clustering model

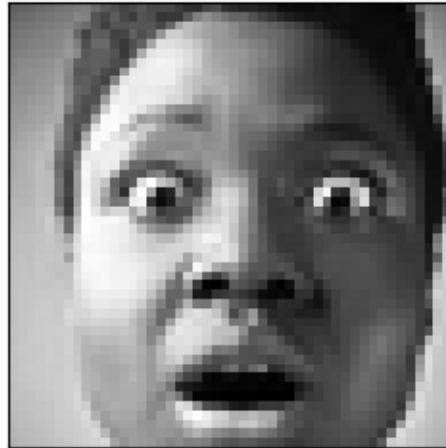
```
[355]: mat_kcl = confusion_matrix(emotion_predictions_kcl, test_set_labels)
sns.heatmap(mat_kcl.T, square=True, annot=True, fmt='d', cbar=False,
            xticklabels= class_names_numbers,
            yticklabels= class_names_numbers)
plt.xlabel('true label')
plt.ylabel('predicted label');
```



Unsurprisingly, the model did not correctly guess the emotion of a randomly chosen image.

```
[357]: i = 7002

plt.figure(figsize=(8,3))
plt.subplot(1,2,1)
plot_image_2(i, emotion_predictions_kcl, test_set_labels, test_set)
```



Happy (Surprised)

Conclusion:

The models varied greatly in their predictive abilities. The convolution neural net had an out of sample accuracy score of 45%. The clustering model had an out of sample accuracy score of 17%, and the random forest model had an out of sample accuracy score of 40%. The neural net was the best model in terms of out of sample accuracy. Because random forests are scalable, can measure relative importance of features, and are less prone to over-fitting than other tree types, I believe that the random forest might still be the best method of the 3 models used here. I think with different activation functions, it's possible that the neural net could far outperform the random forest in terms of validation accuracy, but I think the random forest is still useful for how powerful it is while being so simple and without tweaking. The random forest and the neural net are much better at determining the correct emotions displayed by the images than they would be by pure chance, so the model has some use, but I think that more accuracy is needed in order to justify using this model to make accurate predictions for practical purposes.